RNA_lexis -- User Guide

Authors: Haim Bar (haim.bar@uconn.edu) and Assaf Bester (bestera@technion.ac.il)

Overview

RNA_lexis is an interactive command-line tool for exploring RNA (or DNA) sequence data. Starting from a nucleotide sequence, it automatically identifies repeating subsequences called **xmotifs** and extracts their shorter conserved cores. From there you can visualize the neighbourhood of any core, study k-mer statistics, generate sequence logos, inspect coverage, and compare subsequences -- all from a hierarchical menu driven by numbered choices.

Launching the Application

```
rna_lexis           # after pip install (console script)
rna_lexis_stat      # alias for the same interactive menu
python -m rna_lexis # without installing
```

For batch statistical workflows without entering the interactive menu:

```
rna_lexis_stat_cli score-exact       --fasta FILE --motifs m1 m2 …
rna_lexis_stat_cli rank-cores        --fasta FILE
rna_lexis_stat_cli mutation-families --fasta FILE --motifs m1 m2 …
rna_lexis_stat_cli gapped-motif      --fasta FILE --left LEFT --right RIGHT
```

At startup the tool checks the **last used directory** (remembered from the previous session) for saved session files (`.json`). If no last-used directory is recorded it falls back to the **default data directory** (set by the user in *Change setting*), and then to the current working directory:

- If **exactly one** valid session file is found, it is loaded automatically — no prompt is shown.
- If **more than one** valid session file is found, a numbered menu lets you choose which one to load.
- If **no** valid session file is found, you will be prompted to choose a **data directory** via a folder-browser dialog. The same auto-detection is then applied to the chosen directory.

The last used directory is updated automatically every time a file is opened and is stored in a platform-appropriate config file so it persists across sessions. The default data directory is a user-defined root that is used as the starting point for file dialogs when no last-used directory is set. Both can be viewed in *Show settings* and the default data directory can be changed in *Change setting*.

It is recommended that each sequence analysed be stored in its own directory, since output files are stored in the same directory as the sequence, as well as the saved-session JSON file.

Choosing Input

Every session starts by loading a sequence. Five input methods are available:

```
--- Choose Input ---
1. Load from local file
2. Fetch by Ensembl transcript ID (ENST)
3. Paste sequence
4. Load example dataset (NORAD)
5. Python prompt
```

1 - Load from local file

A file-selector dialog opens. Select any plain-text file containing a nucleotide sequence (letters only; spaces and punctuation are stripped automatically). If you select a **`.json`** session file that was saved by a previous run, that session is restored instantly -- no re-processing required.

FASTA files (`.fa` , `.fasta`) are also supported. The header line (starting with `>`) is used only to detect the format and extract the gene name; it is not included in the sequence. Multi-record FASTA files are handled correctly: all header lines are stripped and the sequence records are concatenated. The gene name from the first header is stored in the session under the key `gene_name` .

2 - Fetch by Ensembl transcript ID

Enter an Ensembl transcript identifier (e.g. `ENST00000456328` or `ENST00000456328.2`). The tool:

1. Checks whether a cached session for that transcript already exists in your data directory; if so, loads it immediately.
2. Otherwise, queries the Ensembl REST API for the cDNA sequence, displays the transcript name and length, and asks you to choose a folder in which to save the data.

The fetched sequence is stored in a JSON file in the chosen directory and the file name contains the transcript number and name (e.g., ENST00000419829_XIST-204.json).

> **Note:** an active internet connection is required for live fetching.

3 - Paste sequence

Paste one or more lines of sequence text directly into the terminal and press **Enter on an empty line** to finish. You will be prompted for a name for the sequence and a folder in which to save results.

4 - Load example dataset (NORAD)

Loads one of two small real datasets bundled with the package itself — no internet connection or account needed, since they ship inside `pip install`:

- **NORAD (human)** — Ensembl transcript `ENST00000565493`, 5401 nt.
- **NORAD (mouse)** — Ensembl transcript `ENSMUST00000192863` (*Norad*), 4945 nt.

NORAD is a good sequence to try the tool on: it contains a well-characterised tandem repeat of the Pumilio Response Element (PRE, core motif `tgtatata`), so running the default discovery pipeline reliably surfaces a real, biologically meaningful repeat structure — useful both for learning the tool and as a sanity check that your installation works correctly. After the sequence loads you're prompted for a folder to save the session to, exactly as with the ENST-fetch option. See `src/rna_lexis/data/README.md` in the package source for full provenance details.

5 - Python prompt

Returns control to the Python interpreter.

Session Management

After loading a sequence the tool automatically:

- **Parses** the text to find xmotifs and cores.

- **Saves a session file** (`<name>.json`) containing the sequence, xmotifs, cores, and summary statistics. On the next run, if this is the only session file in the directory, it is loaded automatically and the parsing step is skipped.
- **Creates a summary CSV** (`<name>_test_init.csv`) with per-sequence statistics (length, occurrence count, mutation rate, stability p-value, and — for RNA/DNA sequences — Markov enrichment and FDR q-values) and opens it in your default spreadsheet application.

To re-open that CSV at any time, use **Open Core file** from the main menu. See below, in the Main Menu section.

Main Menu

```
--- Main Menu ---
 1. Plots
 2. Sequence operations
 3. Open Core file
 4. Summary statistics
 5. Show settings
 6. Change setting
 7. Open User Guide
 8. Clear workspace
 9. Load new input
10. Quit
```

Navigate by typing the option number and pressing **Enter**. Press **Ctrl+D** at any prompt to cancel the current operation and return to the main menu. Press **Ctrl+C** to exit the program immediately.

---

1 - Plots

Opens the Plots submenu:

```
--- Plots ---
1. Core neighbors (detailed)
2. Core neighbors (condensed)
3. K-mers
4. Logo
5. Coverage
6. Motif Match/Mutation
7. Back
```

1.1 Core neighbors (detailed)

Visualises how a chosen core sequence co-occurs with its neighbours across the full text, with one horizontal lane per neighbour.

Prompts:

Prompt	Default	Notes
Sequence to analyse	no default	Must be present in the text
Neighbourhood width	adaptive W(N)	Number of positions on each side of the sequence
Strings to include	2 (xmotifs)	1 = cores, 2 = xmotifs
Plot title	`<file> <sequence>`	Free text
Output file name	*(screen only)*	Leave blank to display interactively
Output format	1 (PNG standard)	1 = PNG, 2 = PNG high-res (3×), 3 = SVG, 4 = HTML
X-axis range	*(full range)*	`min, max` e.g. `100, 500`
Minimum occurrences to include a neighbour	2	Neighbours occurring fewer times than this are excluded
	N	

Prompt	Default	Notes
Show hairpin regions?		Only asked when a companion `*_hairpins.csv` file exists in the session directory

The adaptive default W(N) = clamp(⌊N·ln2 / (2·ñ)⌋, 20, 80), where N is the transcript length and ñ is the median occurrence count of the detected cores. This keeps the null co-occurrence probability near 0.5 regardless of transcript length. The computed value is shown at the prompt; press Enter to accept it or type a different integer.

The interactive HTML plot is always shown on screen. If an output file name is given, the plot is also saved to disk in the chosen format. PNG/SVG export runs with a 45-second timeout; if Kaleido hangs past that, an interactive HTML fallback (`<name>_fallback.html`) and a plain-text error report (`<name>_export_message.txt`) are saved next to it instead.

Reading the plot:

- The reference sequence (s0) sits at **y = 0**.
- **Neighbouring cores** are shown above (y = 1, 2, ...), sorted by how much they overlap with s0. Purple squares = cores that share no positions with s0; green squares/triangles = partial or full overlap.
- **Mutations** of s0 (if any) appear below the axis (y = -1, -2, ...), with the closest mutation to s0 (highest occurrence count) nearest to the axis.
- Triangle direction indicates which end of the neighbour extends beyond s0: **>** extends to the right, **<** extends to the left.
- A legend at the bottom of the plot explains all symbols and colours.

1.2 Core neighbors (condensed)

A compact three-band overview of the same neighbourhood data, designed for long sequences or large neighbour sets where individual lanes would be unreadable.

Prompts: (identical to Core neighbors (detailed))

Prompt	Default	Notes
Sequence to analyse	no default	Must be present in the text
Neighbourhood width	adaptive W(N)	Number of positions on each side of the sequence
Strings to include	2 (xmotifs)	1 = cores, 2 = xmotifs
Plot title	`<file> <sequence>`	Free text
Output file name	*(screen only)*	Leave blank to display interactively
Output format	1 (PNG standard)	1 = PNG, 2 = PNG high-res (3×), 3 = SVG, 4 = HTML
X-axis range	*(full range)*	`min, max` e.g. `100, 500`
Minimum occurrences to include a neighbour	2	Neighbours occurring fewer times than this are excluded
Show hairpin regions?	N	Only asked when a companion `*_hairpins.csv` file exists in the session directory

PNG/SVG export runs with a 45-second timeout; if Kaleido hangs past that, an interactive HTML fallback (`<name>_fallback.html`) and a plain-text error report (`<name>_export_message.txt`) are saved next to it instead.

Reading the plot:

- **s0** (light blue rectangles) sits at **y = 0**.
- All neighbours are collapsed into a single **density strip** at **y = 1**. Opacity scales with the number of distinct neighbours overlapping each position.
- **Green** = neighbour overlaps with or contains s0.
- **Magenta** = neighbour does not overlap with s0.

- **Mutations** of s0 (if any, requires `len(s0) ≥ 6`) are collapsed into a single red **mutation strip** at **y = −1**.
- **Hairpins** (only shown when a companion `*_hairpins.csv` file exists in the session directory) appear as orange bands just below s0.
- The y-axis tick labels report the number of distinct neighbours found.

1.3 K-mers

Analyses the frequency distribution of all substrings of a given length k.

Prompts:

Prompt	Default
k-mer length	5

Then choose a plot type:

```
1. Z-score
2. Robust Z-score
3. Abundance histogram
4. Rank-frequency
5. Return to plots
```

The Z-score plot highlights over/under-represented k-mers. The robust version uses a median-based statistics. For Z-score plots you will also be asked for a **threshold** (default 1.96).

The abundance histogram shows the total number of k-mer occurrences in the text.

The Rank-frequency plot shows the log of the rank of each score vs. the log of the number of occurrences. This plot is often used to check whether the distribution of k-mers follows Zipf's law.

After the plot-specific parameters, you will be prompted for an **output file name** and **format** (PNG standard, PNG high-res 3×, or SVG). Leave the file name blank to display on screen only.

1.4 Logo

Generates a sequence logo centred on all occurrences of a chosen sequence.

Prompts:

Prompt	Default	Notes
Sequence to analyse	no default	Must be present in the text
Bases on each side	0	Flanking context to include
Max mutations	0	Fuzzy matching; capped at len/6
Format	pdf	`pdf` or `svg`
Output file name	`<seq>_logo`	

1.5 Coverage

Shows how much of the text is covered by cores or xmotifs, weighted by a power law.

Prompts:

Prompt	Default	Notes
Strings to include	1 (cores)	1 = cores, 2 = xmotifs
Exponent *a*	1.2	Score = length^a times occurrences
Output file name	*(screen only)*	Leave blank to display interactively
Output format	1 (PNG standard)	1 = PNG, 2 = PNG high-res (3×), 3 = SVG

1.6 Motif Match/Mutation

Plots the positions of up to three user-supplied sequences along the full text as colored rectangles stacked in separate horizontal lanes: sequence 1 at the bottom, sequence 2 above it, and sequence 3 at the top.

Prompts:

Prompt	Default	Notes
Sequence 1 / 2 / 3		Leave blank to stop after fewer than 3
Mutation rate: 1 per N letters	6	Sets the maximum number of mismatches per sequence: `floor(L / N)`
Output file	*(screen only)*	Leave blank to display interactively
Output format	1 (PNG)	1 = PNG, 2 = PNG high-res (3×), 3 = SVG, 4 = HTML
Condense x-axis?	N	If `y`, large empty regions on the x-axis are compressed
Minimum gap size to compress	1000	Only gaps wider than this many positions are compressed (shown only when condensing)

PNG/SVG export runs with a 45-second timeout; if Kaleido hangs past that, an interactive HTML fallback (`<name>_fallback.html`) and a plain-text error report (`<name>_export_message.txt`) are saved next to it instead.

Reading the plot:

- Each sequence occupies its own lane, drawn in a distinct color: **blue** (seq 1, bottom), **orange** (seq 2, middle), **green** (seq 3, top).
- **Exact matches** are solid rectangles.

- **Approximate matches** (within the mutation allowance) carry thin **red vertical lines** at each mismatched character position inside the rectangle.
- The legend shows each sequence with its exact and approximate match counts, plus entries explaining the red line and (if condensed) the dashed separator.
- When condensing is active, gaps exceeding the threshold are replaced by a short stub and a **gray dashed vertical line** marks each compression point. The x-axis ticks show original sequence positions at cluster boundaries.

Optional detailed nucleotide-level view:

After the main plot is shown, you will be asked whether to create a detailed HTML view. If you answer `y`, two more prompts appear:

Prompt	Default	Notes
Position range	full sequence	`start,end` e.g. `200,800`; ranges > 500 nt render slowly
Output HTML file	`<name>_detail.html`	Always saved as HTML; `.html` is appended if omitted

The detail view is a plain HTML page showing every nucleotide in the selected range as monospace text (60 characters per line, prefixed with the genomic position). Hit regions have colored backgrounds matching the main plot. Where two or more sequences overlap, the background is a blended mix of their colors so both are visible simultaneously. Mutation positions are shown in **red uppercase**. A legend lists each sequence with its exact and approximate match counts.

---

2 - Sequence operations

```
--- Sequence operations ---
 1. Find all matches                        ← single-sequence analysis
 2. Search with mutations
```

```
 3. Motif extensions
 4. Print core
 5. Motif spacing / periodicity test
 6. Gapped motif search
 7. Covered area
 8. Core neighbors (text export)

 9. Rank core motifs (Markov/FDR)         ← statistical analysis
10. Mutation-family scoring
11. Alignment score for two sequences
12. K-mer Markov analysis
13. Batch spacing test (all cores & motifs)

14. Export hairpins to CSV                ← other

15. Back
```

2.1 Find all matches

Searches the full text for a query sequence (minimum 4 characters). Use `.` as a single-character wildcard. Reports the number of matches and prints each match with its position in the text.

2.2 Search with mutations

Searches the full text for a single query string and reports both exact and approximate matches (within a configurable mutation allowance). Mismatched characters in approximate matches are printed in **RED** and **UPPERCASE** so differences are immediately visible.

Prompts:

Prompt	Default	Notes
Query string		Blank to cancel and go back to the menu
Mutation rate: 1 per N letters	6	Maximum number of nucleotides per one mismatch

Output:

- **Exact matches** — count and list of start positions.
- **Approximate matches** — each match printed as `pos <position> "<matched string>" dist=<hamming distance>`, with mutated characters highlighted in red uppercase, e.g. `ugaaac G uac`.

Press **Enter** to return to the Sequence operations menu.

2.3 Motif extensions

Combines motif search with pairwise extension analysis. Enter a motif, find all its occurrences (exact or approximate), then automatically compute the longest Hamming-bounded extension for every pair of occurrences and display the results with colour-coded alignments.

Prompts:

Prompt	Default	Notes
Motif		Blank to cancel
Search method	1 (Exact)	`1` = exact (`find_all_matches`); `2` = with mutations (`find_with_mutations`)
Search mutation rate: 1 per N letters	6	Only shown when search method = 2
Extension mutation rate: 1 per N letters	6	Controls how divergent the flanking context may be during extension

Match display:

After the search, all found positions are listed (up to 20):

- **Exact** matches are shown in green with their start position.
- **Approximate** matches (search method 2 only) are shown in red with the Hamming distance.

At least **2 occurrences** (exact or approximate) are required to proceed to the extension step.

Extension output:

Up to 10 pairs are shown, sorted by total extension length (longest first). For each pair:

- Start positions of both seed copies, total extended length, and left/right extension sizes.
- Both extended sequences printed side-by-side (truncated to 80 characters for long extensions, with the full length noted): the seed region is shown in **BOLD UPPERCASE**; mismatched characters in the flanks are shown in **RED UPPERCASE**.
- Hamming distance as a count and percentage.

Printing a pair in full:

After the summary is displayed, you are offered the option to print one pair in full so that the sequences can be selected and copied:

```
Print a pair in full [1–10, blank to skip]:
```

Entering a number prints: - The raw (plain-text) extended sequence for each copy, labelled with its start position — no colour codes, safe to copy directly. - A colour-coded alignment of the same pair (bold seed, red mismatches) for visual reference.

Saving results to CSV:

After the print-in-full step you are prompted to save all pairs to a CSV file:

```
Save all pairs to CSV [Enter for extensions_<motif>.csv, Ctrl+D to skip]:
```

Press **Enter** to save to the default filename (based on the motif), type a custom name, or press **Ctrl+D** to skip. The CSV contains one row per pair with columns: `rank`, `pos1`, `pos2`, `left_ext`, `right_ext`, `total_len`, `hamming`, `ext1`, `ext2`.

2.4 Print core

Given a sequence, finds and prints the xmotifs that contain it as a core.

2.5 Motif spacing / periodicity test

Tests whether the occurrences of a motif are spaced more regularly than random placement would produce. Requires m ≥ 3 occurrences; a warning is shown when m < 6 (low power).

Prompts:

Prompt	Default	Notes
Sequence to test		Blank to cancel
Mutations: 0 = exact only, or N for 1-per-N-nt	0	Enter 6 to allow 1 mutation per 6 nt; maximum N is 6 (rate 1/6)

When a mutation rate > 0 is given, both exact and approximate (Hamming) matches are included. Overlapping positions are deduplicated before the test. The match mode (e.g. "≤1 mutation(s) — 8 exact, 3 approx") is shown in the output header.

Two statistical tests are run:

Gap cluster test (primary) — estimates the candidate period T as the median of the m−1 consecutive gaps, then counts how many gaps fall within $\delta = \max(5, 5\% \cdot T)$ of T. Under the null hypothesis of uniform random placement each gap is Uniform[0, L], giving an expected count of $(m-1) \times 2\delta/L$. The observed count is tested against $\text{Bin}(m-1, 2\delta/L)$; the p-value is Bonferroni-corrected by (m−1) because T is derived from the same data.

Rayleigh test (confirmatory) — maps all m positions modulo T to angles $\theta_i = 2\pi(p_i \bmod T)/T$ on a circle and tests whether they cluster. The mean resultant length $R \in [0,1]$ quantifies concentration (0 = uniform, 1 = all coincide); $Z = m \cdot R^2$ is the test statistic. The Mardia & Jupp (2000) approximation gives the p-value.

Either test reaching p < 0.05 produces a green "Significant" verdict. When gap sizes are mixed, the output automatically notes:

- **Small gaps** (< 15%·T) — likely tandem copies within the same period window.
- **Large gaps** (> 175%·T) — likely one or more periods skipped.

2.6 Gapped motif search

Finds all occurrences of an anchor-gap-anchor pattern `LEFT[gap:min–max]RIGHT` and scores the whole family under the Markov background.

Prompts:

Prompt	Default	Notes
Left anchor sequence		Blank to cancel
Right anchor sequence		Blank to cancel
Minimum gap length	0	
Maximum gap length	30	
Output CSV file	`<session>_gapped_motif.csv`	

Output CSV columns:

Column	Description
`pattern`	Pattern string, e.g. `ACGU[gap:0-30]UGCA`
`observed_count`	Number of pattern hits found
`expected_markov`	Expected count under the Markov null
`enrichment_markov`	`observed / expected`
`p_markov`	Poisson upper-tail p-value
`start`	Start position of each hit (one row per hit)

Column	Description
`gap_length`	Gap length for this hit
`matched_sequence`	Full matched sequence including the gap

The family-level statistics (`pattern` , `observed_count` , `expected_markov` , `enrichment_markov` , `p_markov`) are the same on every row; per-hit columns (`start` , `gap_length` , `matched_sequence`) vary by row. Up to 20 hits are printed to the terminal.

2.7 Covered area

Computes and prints the coverage score for a single sequence: `length^a times occurrences` , where *a* is a configurable exponent (default 1.2, matching the session setting).

2.8 Core neighbors (text export)

Exports the sequence regions associated with a query core and its neighbourhood as a CSV file, so that individual region sequences can be copied directly into other tools (e.g. **Alignment score for two sequences**, **Search with mutations**).

How regions are defined:

Every occurrence of s0, its single-nucleotide mutations, and all its neighbours (sequences with a positive conditional-probability score within the window) is expanded by ±wd nucleotides. Overlapping expanded intervals are merged into contiguous regions. The result is a set of non-overlapping windows that cover every area of the text where any of the tracked sequences cluster together — including regions where neighbours bridge the gap between two s0 occurrences, and regions that contain only neighbours with no s0 nearby.

Prompts:

Prompt	Default	Notes
	no default	

Prompt	Default	Notes
Sequence to analyse		Must be present in the text
Neighbourhood width	adaptive W(N)	Half-window used for interval expansion and gap merging
Strings to include	2 (xmotifs)	1 = cores, 2 = xmotifs
X-axis range	*(full range)*	`min, max` e.g. `100, 500` — limits which positions are considered
Output CSV file	`<session>_<seq>_regions.csv`	In the session directory; leave blank to use the default

Output CSV columns:

Column	Description
`start`	0-based start position of the region in the full sequence
`end`	0-based end position (inclusive)
`seq`	`txt[start:end+1]` — the raw nucleotide sequence of the region

Results are also printed to the terminal. The CSV is opened automatically in your default application when writing completes.

2.9 Rank core motifs (Markov/FDR)

Enumerates all shared substrings of the current xmotifs within a configurable length range, scores each candidate against a transcript-

specific Markov background, and saves a ranked CSV. Candidates are accepted as *statistically supported* when the FDR-corrected q-value is below the threshold **and** the enrichment ratio exceeds the minimum enrichment.

Prompts:

Prompt	Default	Notes
Candidate minimum core length	5	Shortest substring to test
Candidate maximum core length	18	Longest substring to test
Minimum xmotif-type support	2	Candidate must appear in at least this many xmotif families
Minimum Markov enrichment	10	`observed / expected` ratio threshold
FDR q-value threshold	0.05	Benjamini–Hochberg threshold
Output CSV file	`<session>_ranked_cores_markov.csv`	

Output CSV columns:

Column	Description
`motif`	Candidate core sequence
`exact_count`	

Column	Description
	Number of exact occurrences in the transcript
`expected_markov`	Expected count under the Markov null
`enrichment_markov`	`observed / expected` ratio
`p_markov`	Poisson upper-tail p-value
`q_markov`	BH-adjusted p-value
`statistically_supported`	`True` if q and enrichment thresholds are met
`coverage_bp`	Base-pairs covered by non-overlapping occurrences
`xmotif_type_support`	Number of distinct xmotif families containing this candidate
`rank_statistical`	Primary rank (statistical support first)
`rank_coverage`	Secondary rank (coverage)

Up to 15 supported candidates are printed to the terminal after the CSV is saved.

2.10 Mutation-family scoring

Tests one or more motifs at every Hamming radius allowed by the mutation cap. For each motif and radius, the complete neighbourhood — all sequences within that Hamming distance — is counted and scored against the Markov background. The best-supported radius per motif is written to a separate `_best.csv` file.

Prompts:

Prompt	Default	Notes
Motif source	1	1 = enter a motif, 2 = current cores,

Prompt	Default	Notes
		3 = current xmotifs
Motif		Only shown when source = 1; blank to cancel
Mutation search cap: 1 per N letters	6	Sets the maximum Hamming radius tested
Minimum family enrichment	5	Enrichment threshold for the full Hamming neighbourhood
Maximum expected family count	5	Families with a higher expected count are not scored
FDR q-value threshold	0.05	BH threshold applied across all (motif, radius) pairs
Output CSV file	`<session>_mutation_family_tests.csv`	Full results

Two files are written: - **Full results** (`_mutation_family_tests.csv`): one row per (motif, radius) pair. - **Best radius** (`_mutation_family_tests_best.csv`): one row per motif showing the strongest accepted radius.

Decision values in `_best.csv` :

Decision	Meaning
`mutation_supported`	Family is statistically enriched at this radius
`exact_or_specific_only`	Only the exact sequence (radius 0) is enriched
`below_threshold`	No radius passes the enrichment + FDR criteria

2.11 Alignment score for two sequences

Aligns two substrings extracted from the loaded text by position.

Prompts:

Prompt	Notes
Start position of first sequence	0-based index into the text
Start position of second sequence	
Sequence length	Both substrings share the same length
Alignment type	1 = Local (Smith–Waterman), 2 = Global (Gotoh)

The alignment is printed with gap symbols and the following scores:

- **Raw score** — the integer alignment score.
- **Normalised score** — raw score divided by the self-alignment score of the shorter sequence; lies in [0, 1]. Values above 0.70 indicate a clearly related pair; above 0.90 near-identical.
- **Bit score** — length-independent score computed with the Karlin–Altschul formula ($\lambda = 1.28$, $K = 0.46$); comparable across alignments of different lengths.
- **E-value** — expected number of alignments with a score this high or better by chance, using the full transcript as the reference database. Values below 10^{-3} are considered significant.

2.12 K-mer Markov analysis

Computes analytical p-values for every observed k-mer using a Markov-model null — no shuffling or random seeds required. For each k-mer the expected count is derived from the Prum/Schbath formula conditioned on shorter k-mer frequencies; the observed count is then tested against a Poisson(expected) null. Two one-sided p-values are reported per k-mer, testing for over- and under-representation separately. All results are saved to a CSV file sorted by the more extreme of the two p-values.

Markov background order controls how much context the null model conditions on:

Order	Null conditions on	Typical use
0	Single-nucleotide (A/C/G/U) frequencies	Equivalent to the old shuffle null
1 *(default)*	Dinucleotide frequencies	Recommended for sequences of a few kb
2	Trinucleotide frequencies	Longer sequences or k-mers ≥ 10

> **Note:** Using order = k−1 (the classic Prum/Schbath setting) is appropriate only for very long sequences. For short transcripts (< ~10 kb) it makes expected ≈ observed for long k-mers, producing no significant results. Order 1 is almost always the right choice.

Prompts:

Prompt	Default	Notes
k-mer length	6	Length of the substrings to count (≥ 2)

Prompt	Default	Notes
Markov background order	1	0 = nucleotide, 1 = dinucleotide, … (max k−2)
Output CSV file	`<name>_kmer<k>_order<order>_markov_pvalues.csv`	Path to the results file

Output CSV columns:

Column	Description
`kmer`	The k-mer sequence
`real_count`	Number of times it appears in the loaded sequence
`expected_count`	Expected count under the Markov null
`pvalue_over`	P(X ≥ obs) under Poisson(expected) — small = **over-represented**
`evalue_over`	`pvalue_over × m` (expected false positives among *m* k-mers)
`pvalue_over_bh`	BH-adjusted p-value for over-representation (FDR)
`pvalue_under`	P(X ≤ obs) under Poisson(expected) — small = **under-represented**
`evalue_under`	`pvalue_under × m`
`pvalue_under_bh`	BH-adjusted p-value for under-representation (FDR)
`direction`	`'over'` or `'under'` — which effect is stronger

Rows are sorted by `min(pvalue_over, pvalue_under)` so the most extreme k-mers appear first. BH correction is applied separately within each family of *m* tests.

2.13 Batch spacing test (all cores & motifs)

Runs the spacing / periodicity test (gap cluster + Rayleigh) on every core and xmotif in the current session simultaneously and saves a ranked CSV. Sequences with fewer than 3 occurrences are skipped automatically.

Prompts:

Prompt	Default	Notes
Mutations: 0 = exact only, or N for 1-per-N-nt	0	Same rate as the single-sequence spacing test; maximum N is 6
Output CSV file	`<session>_spacing_batch.csv`	

Each sequence is tagged as `core`, `xmotif`, or `both` depending on which list it appears in (the union is tested, with duplicates counted once). Results are sorted by `p_cluster` ascending so the most periodically spaced sequences appear first. The top 10 are printed to the terminal; all results are saved to the CSV.

Output CSV columns:

Column	Description
`motif`	Sequence tested
`type`	`core`, `xmotif`, or `both`
`m`	Number of occurrences used
`period`	Candidate period T (median gap)
`delta`	Tolerance window δ = max(5, 5%·T)
`k_near_T`	Gaps within [T−δ, T+δ]

Column	Description
`n_gaps`	Total consecutive gaps (m − 1)
`p_cluster`	Bonferroni-corrected Binomial p-value (primary sort key)
`rayleigh_r`	Mean resultant length R
`p_rayleigh`	Rayleigh p-value
`significant`	`True` if either p < 0.05

2.14 Export hairpins to CSV

Exports all detected hairpin regions for the loaded sequence to a CSV file. Each row contains the start position, end position, stem sequence, loop sequence, and full hairpin sequence.

---

3 - Open Core file

Opens (or regenerates) the summary CSV (`<name>_test_init.csv`) in your default spreadsheet application. If the file already exists and contains the statistical columns, it is opened directly. If it is missing or was created by an older version (and lacks the statistical columns), it is regenerated automatically. If the file is already open in a spreadsheet application, the existing window is brought to the foreground rather than opening a second copy.

---

4 - Summary statistics

Displays counts and lengths derived from the current session:

Statistic	Description
Text length	Total number of characters in the sequence

Statistic	Description
Number of xmotifs	Count of recurring subsequences found
Longest / Shortest xmotif	Character lengths
Number of cores	Count of conserved core sequences
Longest / Shortest core	Character lengths

Press **Enter** to return to the main menu.

---

5 - Show settings

Prints the current default values (read-only) and the persistent preferences (`default_data_dir`, `last_used_dir`). Press **Enter** to return.

---

6 - Change setting

Lets you change the parameters used when parsing the sequence. Pressing Enter at any prompt keeps the current value.

Parameter	Default	Description
Min xmotif length	7	Shortest subsequence considered as an xmotif
Max xmotif length	40	Longest subsequence considered as an xmotif
Min core length	6	Shortest conserved core
Min occurrences	2	Minimum number of times a pattern must appear
Coverage weight (*pwr*)	1.2	Exponent for the length times occurrence score

Parameter	Default	Description
Clear screen	True	Whether to clear the terminal between menus
Session data directory	*(current session)*	Folder used for file dialogs in this session
Default data directory	*(not set)*	Persistent root folder for file dialogs; saved across sessions. Enter to keep, `-` to clear, `B` to browse.

Note: changing min/max xmotif length, min core length, or min occurrences triggers an automatic re-analysis of the loaded text. The session JSON and summary CSV are updated automatically.

Auto-expansion of max xmotif length: if the longest xmotif found during analysis equals the current maximum length, the maximum is automatically raised by 20 and the analysis is re-run. This repeats until the longest xmotif is strictly shorter than the maximum. The expanded value is kept as the new maximum for the session and is shown in *Show settings*.

---

7 - Open User Guide

Opens this user guide in your default web browser.

---

8 - Clear workspace

Deletes all generated files from the working directory while preserving the loaded input file and any other files with a FASTA extension (`.fasta`, `.fa`, `.fna`, `.fas`).

Before anything is deleted the menu displays two lists:

- **Files that will be DELETED** — session JSON, summary CSV, plot files, and any other output generated by a previous run.

- **Files that will be KEPT** — the loaded input file (regardless of its extension) and any additional FASTA files found in the same directory.

You must type `YES` (exact, case-sensitive) and press **Enter** to confirm. Pressing Enter without typing `YES`, or typing anything else, cancels the operation and no files are touched.

If the deletion succeeds, the session is reset and the tool returns to the **Choose Input** screen so you can reload the original sequence fresh with a clean directory.

---

9 - Load new input

Discards the current session and returns to the **Choose Input** screen so you can work with a different sequence.

---

10 - Quit

Exits the program.

Tips

- **Resuming a session:** launch the tool from the directory that contains the session file (`.json`). If exactly one valid session file is present, it is detected and loaded automatically with no prompts. If multiple session files are present, a selection menu is shown.
- **Cancelling mid-prompt:** press **Ctrl+D** to abandon the current operation and jump back to the main menu without losing your session.
- **Output files:** plots and logos are saved to the working directory (the folder of the loaded file) unless you specify an absolute path.

- **RNA vs DNA:** if the sequence contains only `a`, `c`, `g`, `u` (RNA), uracil (`u`) is automatically converted to thymine (`t`) for internal processing.

Troubleshooting

Issues specific to using the interactive menu. For installation and dependency problems (missing `tkinter`, broken image export, etc.), see the **Troubleshooting** section in `README.md` instead.

- **"Sequence not found" for a sequence you can see in your data.** For RNA sessions the loaded text is stored internally with `u` converted to `t` (see *RNA vs DNA* above) — but that conversion is **not** applied to what you type at a search prompt. Typing `ugcaug` against an RNA session will find nothing; type `tgcaug` instead. This affects every "enter a sequence/motif" prompt (Find all matches, Core neighbors, Motif extensions, etc.).
- **A Core neighbors plot looks sparser than expected, or a neighbour you know is there doesn't show up.** Check the **minimum occurrences** prompt (default 2 — a neighbour occurring only once is excluded) and the **X-axis range** prompt (default full range, but a previously-entered narrow range persists until changed). Mutations are always shown regardless of the minimum-occurrences filter.
- **`Unexpected error: ...` appears and you're dropped back to the main menu.** This is a catch-all error handler — it means the specific action you just tried failed, not that your session was lost. Your loaded sequence, xmotifs, and cores are still in memory; you can retry the same operation or try a different one. If it happens repeatedly for the same action, note the exact error text before reporting it.
- **The tool keeps asking you to choose a data directory instead of loading your session automatically.** This happens when the remembered last-used directory (see *Show settings*) no longer contains a valid session file — e.g. the folder was moved, renamed, or the session `.json` / `_test_init.csv` pair was deleted. Browse to

the correct folder once; it will be remembered for next time. If a folder contains **more than one** valid session, you'll be asked to pick which one every time — this is expected, not a bug.

- **A statistical test or extension refuses to run ("need ≥ 2 occurrences", "need ≥ 3 occurrences").** These are hard minimums, not defaults: Motif extensions and Extend-style pairing need at least 2 exact/approximate occurrences; the spacing/periodicity test needs at least 3. Try a shorter or less specific motif, or allow mutations (search method 2) to pick up more occurrences.
- **Changing a setting (min/max xmotif length, min core length, min occurrences) seems to hang or re-run several times.** If the longest xmotif found equals the current maximum length, RNA_lexis automatically raises the maximum by 20 and re-analyses — repeating until the longest xmotif is strictly shorter than the maximum (see *Change setting* above). On long, repeat-rich sequences this can take a few passes; let it finish rather than interrupting.

Package Structure

The `rna_lexis` package is organised into focused sub-modules. New code should import directly from the relevant module rather than from the top-level `rna_lexis` namespace. For a full API reference (function signatures, data structures, worked examples) rather than the interactive-menu walkthrough this guide covers, see llms.txt in the repository root.

Module	Contents
`rna_lexis.algorithms`	Pure-computation functions — k-mer counting, motif finding, core extraction, hairpin generation, fuzzy matching, alignment scoring helpers. No I/O or plotting dependencies.
`rna_lexis.alignment`	Gotoh global and local alignment with affine gap penalties (`gotoh_global` ,

Module	Contents
	`gotoh_local`, `print_alignment`, `AlignmentResult`).
`rna_lexis.plots`	All matplotlib and Plotly visualisation functions — sequence logos, z-score plots, k-mer histograms, frequency-rank plots, coverage plots, sequence-hit diagrams.
`rna_lexis.io`	File and session I/O (`read_text`, `save_session`, `load_session`, `is_valid_session`, `init_summary`), Ensembl REST API fetching (`fetch_enst_cdna`), and platform-aware file opening utilities.
`rna_lexis.statistical`	Transcript-specific Markov background scoring, Poisson/FDR motif enrichment tests, mutation-family scoring, and gapped-motif search. Key functions: `score_exact_motifs`, `rank_core_candidates`, `mutation_family_tests`, `best_mutation_family_per_motif`, `find_gapped_motif_hits`, `score_gapped_motif`.
`rna_lexis.dialogs`	Thin tkinter wrappers for native file/directory chooser dialogs (`openFile`, `openDir`).
`rna_lexis.menu`	Interactive CLI menu — all user-facing prompts, menus, and session management.
`rna_lexis.test_cli`	Batch command-line interface for statistical workflows. Entry point: `rna_lexis_stat_cli`. Sub-commands: `score-exact`, `rank-cores`, `mutation-families`, `gapped-motif`.

Module	Contents
`rna_lexis.RNAlang`	Backward-compatibility shim. Re-exports every public symbol from the modules above so that existing code using `from rna_lexis.RNAlang import …` or `import rna_lexis.RNAlang as RNA` continues to work without modification.

Example — using sub-modules directly

```
from rna_lexis.algorithms import cores, find_with_mutations
from rna_lexis.alignment import gotoh_global, print_alignment
from rna_lexis.io import read_text, save_session

txt = read_text("my_sequence.txt")
corelist = cores(txt)
result = gotoh_global(corelist[0], corelist[1])
print_alignment(result)
```